



Design Patterns and Unit Testing

Names:	Molar, Jabez C. Oabel, Edmarc Justin C.	Date:	September 12, 2026
Program:	CPE		

A. Learning Outcomes:

At the end of this laboratory exercise, students should be able to:

- Apply software design patterns in application development.
- Implement the Singleton and Factory Design Patterns.
- Create unit tests using a testing framework.
- Validate software functionality through automated testing.
- Collaborate effectively in a team-based software development project.

B. Scenario:

Paws and Care Veterinary Clinic currently manages pet appointments manually, causing scheduling conflicts and difficulty tracking records. To improve efficiency, the clinic wants a Vet Clinic Appointment Management System that allows staff to register pet owners, manage pet records, schedule appointments, and cancel bookings. The system will utilize the Singleton Pattern for managing a single clinic database, the Factory Pattern for creating different pet types, and Unit Testing to ensure all functionalities work correctly.

C. Functional Requirements

1. Pet Owner Management
 - Register new pet owners.
 - View registered pet owners.
 - Store owner information (Owner ID, Name, Contact Number).
 2. Pet Management
 - Add pet records.
 - View pet information.
 - Associate pets with their respective owners.
 - Support different pet types:
 - Dog
 - Cat
 - Bird
 - Rabbit
 3. Appointment Management
 - Schedule appointments.
 - View appointment schedules.
 - Cancel appointments.
 - Update appointment status (Scheduled, Completed, Cancelled).
 4. Design Pattern Implementation
 - Singleton Pattern
 - Create a single ClinicDatabase instance to manage all records.
 - Factory Pattern
 - Create pet objects through a PetFactory class.
 5. Unit Testing
- Develop unit tests for:
- Register Pet Owner
 - Add Pet Record
 - Schedule Appointment
 - Cancel Appointment
 - Validate Singleton Instance



Design Patterns and Unit Testing

(Please include the following: **1.** Screenshots of the code **2.** Screenshots of the output)

Duplicate if necessary

Screenshots of Code (Name of Members) - Molar, Oabel
--

Laboratory Report 4

Design Patterns and Unit Testing

```
database.py clinic_data.json
1 import threading
2 import itertools
3
4
5 class ClinicCollection(dict):
6     def __getitem__(self, key):
7         if isinstance(key, int):
8             values = list(self.values())
9             if 0 <= key < len(values):
10                 return values[key]
11             raise IndexError("list index out of range")
12         return super().__getitem__(key)
13
14 import json
15 import os
16 from pet_factory import PetFactory
17
18 class ClinicDatabase:
19     _instance = None
20     _lock = threading.Lock()
21     _storage_file = "clinic_data.json"
22
23     def __new__(cls):
24         with cls._lock:
25             if cls._instance is None:
26                 cls._instance = super(ClinicDatabase, cls).__new__(cls)
27                 cls._instance._initialize()
28             return cls._instance
29
30     def _initialize(self):
31         self._owners = ClinicCollection()
32         self._pets = ClinicCollection()
33         self._appointments = ClinicCollection()
34         self._owner_counter = itertools.count(1)
35         self._pet_counter = itertools.count(1)
36         self._apt_counter = itertools.count(1)
37         self._owners = {}
38         self._pets = {}
39         self._appointments = {}
40         self._owner_counter = itertools.count(1)
41         self._pet_counter = itertools.count(1)
42         self._apt_counter = itertools.count(1)
43         self._load_from_disk()
44
45     @classmethod
46     def get_instance(cls):
47         if cls._instance is None:
48             cls()
49         return cls._instance
50
51     @classmethod
52     def reset_instance(cls):
53         with cls._lock:
54             cls._instance = None
55
56     def _save_to_disk(self):
57         serialized_pets = {}
58         for pid, pet in self._pets.items():
59             serialized_pets[pid] = {
60                 "pet_id": pet.pet_id,
61                 "pet_type": pet.pet_type,
62                 "name": pet.name,
63                 "owner_id": pet.owner_id
64             }
65
66         data = {
67             "owners": self._owners,
68             "pets": serialized_pets,
69             "appointments": self._appointments
70         }
71         with open(self._storage_file, "w") as f:
72             json.dump(data, f, indent=4)
73
74     def _load_from_disk(self):
75         if not os.path.exists(self._storage_file):
76             return
77
78         try:
79             with open(self._storage_file, "r") as f:
80                 data = json.load(f)
81
82             self._owners = data.get("owners", {})
83             self._appointments = data.get("appointments", {})
84
85             # Reconstruct pet objects using PetFactory
86             raw_pets = data.get("pets", {})
87             for pid, pinfo in raw_pets.items():
88                 pet_obj = PetFactory.create_pet(
89                     pinfo["pet_id"],
90                     pinfo["pet_type"],
91                     pinfo["name"],
92                     pinfo["owner_id"]
93                 )
94                 self._pets[pid] = pet_obj
95
96             # Re-sync auto-increment counters to prevent ID collisions
97             self._sync_counters()
98
99         except (json.JSONDecodeError, KeyError):
100             pass
101
102     def _sync_counters(self):
103         owner_ids = [int(k.split("-")[1]) for k in self._owners.keys() if k.startswith("OWN-")]
```

Laboratory Report 4

Design Patterns and Unit Testing

```
104     max_owner = max(owner_ids, default=0)
105     self._owner_counter = itertools.count(max_owner + 1)
106
107     pet_ids = [int(k.split("-")[1]) for k in self._pets.keys() if k.startswith("PET-")]
108     max_pet = max(pet_ids, default=0)
109     self._pet_counter = itertools.count(max_pet + 1)
110
111     apt_ids = [int(k.split("-")[1]) for k in self._appointments.keys() if k.startswith("APT-")]
112     max_apt = max(apt_ids, default=0)
113     self._apt_counter = itertools.count(max_apt + 1)
114
115     def generate_owner_id(self):
116         return f"OWN-{next(self._owner_counter):04d}"
117
118     def generate_pet_id(self):
119         return f"PET-{next(self._pet_counter):04d}"
120
121     def generate_appointment_id(self):
122         return f"APT-{next(self._apt_counter):04d}"
123
124     def register_owner(self, owner_id_or_name, name=None, contact=None):
125         if contact is None:
126             if name is None:
127                 raise TypeError("register_owner requires either (name, contact) or (owner_id, name, contact)")
128             name, contact = owner_id_or_name, name
129             owner_id = self.generate_owner_id()
130         else:
131             owner_id = str(owner_id_or_name).strip()
132             if not owner_id:
133                 raise ValueError("Owner ID cannot be empty.")
134
135     def register_owner(self, name, contact):
136         clean_name = str(name).strip()
137         clean_contact = str(contact).strip()
138
139         if not clean_name or not clean_contact:
140             raise ValueError("Owner Name and Contact Number cannot be empty.")
141
142         if contact is None:
143             owner_id = self.generate_owner_id()
144
145         owner_id = self.generate_owner_id()
146         self._owners[owner_id] = {
147             "owner_id": owner_id,
148             "name": clean_name,
149             "contact": clean_contact
150         }
151         self._save_to_disk()
152         return owner_id
153
154     def get_owners(self):
155         return self._owners
156
157     def add_pet(self, pet):
158         clean_owner_id = str(pet.owner_id).strip()
159         if clean_owner_id not in self._owners:
160             raise KeyError(f"Cannot add pet: Owner ID '{clean_owner_id}' does not exist.")
161         self._pets[pet.pet_id] = pet
162         self._save_to_disk()
163
164     def get_pets(self):
165         return self._pets
166
167     def schedule_appointment(self, appointment_id_or_pet_name, pet_name=None):
168         if pet_name is None:
169             clean_pet_name = str(appointment_id_or_pet_name).strip()
170             if not clean_pet_name:
171                 raise ValueError("Pet Name cannot be empty.")
172             apt_id = self.generate_appointment_id()
173         else:
174             apt_id = str(appointment_id_or_pet_name).strip()
175             if not apt_id:
176                 raise ValueError("Appointment ID cannot be empty.")
177             clean_pet_name = str(pet_name).strip()
178             if not clean_pet_name:
179                 raise ValueError("Pet Name cannot be empty.")
180
181     def schedule_appointment(self, pet_name):
182         clean_pet_name = str(pet_name).strip()
183         if not clean_pet_name:
184             raise ValueError("Pet Name cannot be empty.")
185
186         apt_id = self.generate_appointment_id()
187         self._appointments[apt_id] = {
188             "appointment_id": apt_id,
189             "pet_name": clean_pet_name,
190             "status": "Scheduled"
191         }
192         self._save_to_disk()
193         return apt_id
194
195     def update_appointment_status(self, appointment_id, status):
196         clean_apt_id = str(appointment_id).strip()
197         valid_statuses = {"Scheduled", "Completed", "Cancelled"}
198         formatted_status = str(status).strip().capitalize()
199
200         if formatted_status not in valid_statuses:
201             raise ValueError(f"Invalid status '{status}'. Allowed values: {' '.join(sorted(valid_statuses))}")
```

Laboratory Report 4

Design Patterns and Unit Testing

```
202
203
204     if clean_appt_id in self._appointments:
205         self._appointments[clean_appt_id]["status"] = formatted_status
206         self._save_to_disk()
207         return True
208     return False
209
210 def cancel_appointment(self, appointment_id):
211     return self.update_appointment_status(appointment_id, "Cancelled")
212
213 def get_appointments(self):
214     return self._appointments
215
216 def clear(self):
217     self._owners.clear()
218     self._pets.clear()
219     self._appointments.clear()
220     self._owner_counter = itertools.count(1)
221     self._pet_counter = itertools.count(1)
222     self._appt_counter = itertools.count(1)
223     self._appt_counter = itertools.count(1)
224     if os.path.exists(self._storage_file):
225         os.remove(self._storage_file)
```

Laboratory Report 4

Design Patterns and Unit Testing

```

1 # main.py
2
3 def read_required_input(prompt, field_name):
4     while True:
5         value = input(prompt)
6         if value and value.strip():
7             return value
8         print(f"{field_name} cannot be empty. Please press a valid value and then Enter.")
9
10
11 def run_tests():
12     print("\n" + "-" * 40)
13     print("Running Automated Unit Tests")
14     print("-" * 40)
15     suite = unittest.defaultTestLoader.loadTestsFromModule(test_clinic)
16     runner = unittest.TextTestRunner(verbosity=2)
17     runner.run(suite)
18
19
20 def main():
21     db = ClinicDatabase.get_instance()
22
23     while True:
24         print("\n=== Veterinary Clinic Management System ===")
25         print("a. Pet Owner Management")
26         print("b. Pet Management (Factory Implementation)")
27         print("c. Appointment Management")
28         print("d. Run Unit Tests")
29         print("e. Exit")
30
31         choice = read_menu_choice("\nEnter your choice: ")
32         if choice == "":
33             continue
34
35         if choice == "a":
36             print("\nPet Owner Management")
37             print("a. Register Owner")
38             print("b. View Owners")
39             sub = read_menu_choice("Select option: ")
40             if sub == "":
41                 continue
42
43             if sub == "a":
44                 name = read_required_input("Enter Owner Name: ", "Owner Name")
45                 contact = read_required_input("Enter Contact Number: ", "Contact Number")
46                 try:
47                     generated_id = db.register_owner(name, contact)
48                     print(f"Owner '{name}' registered successfully with Auto-Generated ID: {generated_id}")
49                 except ValueError as e:
50                     print(f"Error: {e}")
51             elif sub == "b":
52                 owners = db.get_owners()
53                 print("\nRegistered Owners:")
54                 if not owners:
55                     print("No registered owners found.")
56                 else:
57                     for owner_id, info in owners.items():
58                         print(f"Owner ID: {owner_id} | Name: {info['name']} | Contact: {info['contact']}")
59
60         elif choice == "b":
61             print("\nPet Management")
62             print("a. Add Pet Record")
63             print("b. View Pets")
64             sub = read_menu_choice("Select option: ")
65             if sub == "":
66                 continue
67
68             if sub == "a":
69                 ptype = read_required_input("Enter pet type (Dog, Cat, Bird, Rabbit): ", "Pet type")
70                 pname = read_required_input("Enter pet name: ", "Pet name")
71                 oid = read_required_input("Enter Owner ID: ", "Owner ID")
72                 try:
73                     pet_id = db.generate_pet_id()
74                     pet_obj = PetFactory.create_pet(pet_id, ptype, pname, oid)
75                     db.add_pet(pet_obj)
76                     print(f"Successfully created and associated {pet_obj.pet_type} '{pet_obj.name}' (ID: {pet_obj.pet_id}) with Owner {oid}")
77                 except (ValueError, KeyError) as e:
78                     print(f"Error: {e}")
79             elif sub == "b":
80                 pets = db.get_pets()
81                 print("\nRegistered Pets:")
82                 if not pets:
83                     print("No registered pets found.")
84                 else:
85                     for pet_id, pet in pets.items():
86                         print(f"Pet ID: {pet.pet_id} | Type: {pet.pet_type} | Name: {pet.name} | Owner ID: {pet.owner_id} | Sound: {pet.speak()}")
87
88         elif choice == "c":
89             print("\nAppointment Management")
90             print("a. Schedule Appointment")
91             print("b. View Appointments")
92             print("c. Update/Cancel Appointment")
93             sub = read_menu_choice("Select option: ")
94             if sub == "":
95                 continue
96
97             if sub == "a":
98                 pet_name = read_required_input("Enter Pet Name: ", "Pet Name")
99                 try:
100                     apt_id = db.schedule_appointment(pet_name)
101                     print(f"Appointment scheduled successfully with Auto-Generated ID: {apt_id}")
102                 except ValueError as e:
103                     print(f"Error: {e}")
104             elif sub == "b":
105                 appointments = db.get_appointments()
106                 print("\nScheduled Appointments:")
107                 if not appointments:
108                     print("No scheduled appointments found.")
109                 else:
110                     for apt_id, info in appointments.items():
111                         print(f"Appointment ID: {apt_id} | Pet Name: {info['pet_name']} | Status: {info['status']}")
112             elif sub == "c":
113                 apt_id = read_required_input("Enter Appointment ID to update: ", "Appointment ID")
114                 status = read_required_input("Enter status (Scheduled, Completed, Cancelled): ", "Status")
115                 try:
116                     if db.update_appointment_status(apt_id, status):
117                         print(f"Appointment {apt_id} status updated to {status.capitalize()}.")
118                     else:
119                         print("Appointment ID not found.")
120                 except ValueError as e:
121                     print(f"Error: {e}")
122
123         elif choice == "d":
124             run_tests()
125
126         elif choice == "e":
127             print("\nExiting application...")
128             sys.exit(0)
129
130         else:
131             print("\nInvalid option. Please try again.")
132
133 if __name__ == "__main__":
134     main()
135
```

Laboratory Report 4

Design Patterns and Unit Testing

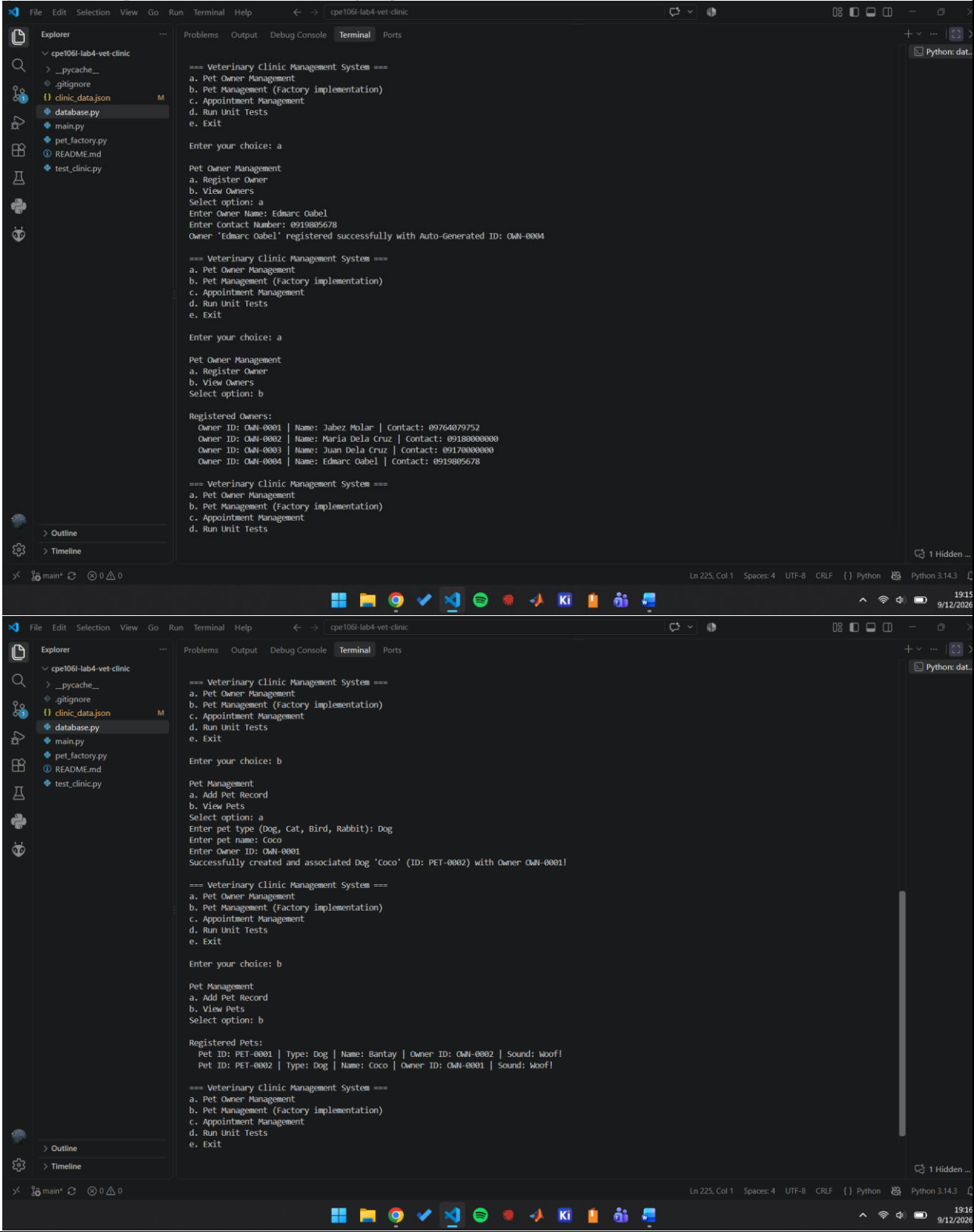
```
+ pet_factory.py
1 from abc import ABC, abstractmethod
2
3
4 class Pet(ABC):
5     def __init__(self, pet_id, pet_type, name, owner_id):
6         self.pet_id = str(pet_id).strip() if pet_id is not None else ""
7         self.pet_type = pet_type
8         self.name = name.strip()
9         self.owner_id = str(owner_id).strip()
10
11     @abstractmethod
12     def speak(self):
13         pass
14
15     def __repr__(self):
16         return f"({self.pet_type})(Name='{self.name}', OwnerID='{self.owner_id}', Sound='{self.speak()}')"
17
18
19 class Dog(Pet):
20     def __init__(self, name, owner_id, pet_id=None):
21         super().__init__(pet_id, "Dog", name, owner_id)
22
23     def speak(self):
24         return "woof!"
25
26
27 class Cat(Pet):
28     def __init__(self, name, owner_id, pet_id=None):
29         super().__init__(pet_id, "Cat", name, owner_id)
30
31     def speak(self):
32         return "Meow!"
33
34
35 class Bird(Pet):
36     def __init__(self, name, owner_id, pet_id=None):
37         super().__init__(pet_id, "Bird", name, owner_id)
38
39     def speak(self):
40         return "Chirp!"
41
42
43 class Rabbit(Pet):
44     def __init__(self, name, owner_id, pet_id=None):
45         super().__init__(pet_id, "Rabbit", name, owner_id)
46
47     def speak(self):
48         return "Squeak!"
49
50
51 class PetFactory:
52     _pet_registry = {
53         "dog": Dog,
54         "cat": Cat,
55         "bird": Bird,
56         "rabbit": Rabbit
57     }
58
59     @classmethod
60     def create_pet(cls, *args):
61         if len(args) == 3:
62             pet_type, name, owner_id = args
63             pet_id = None
64         elif len(args) == 4:
65             pet_id, pet_type, name, owner_id = args
66         else:
67             raise TypeError("create_pet expects either (pet_type, name, owner_id) or (pet_id, pet_type, name, owner_id)")
68
69         normalized = str(pet_type).strip().lower()
70         if normalized not in cls._pet_registry:
71             raise ValueError(f"Pet type '{pet_type}' is not supported.")
72
73         pet_class = cls._pet_registry[normalized]
74         return pet_class(name, owner_id, pet_id=pet_id)
75
76
77 + database.py
78
79 + test_clinic.py
80
81 test_clinic.py > ...
82 1 import unittest
83 2 from database import ClinicDatabase
84 3 from pet_factory import PetFactory, Dog, Cat
85 4
86 5 class TestClinicManagement(unittest.TestCase):
87 6     def setUp(self):
88         ClinicDatabase.reset_instance()
89         self.db = ClinicDatabase.get_instance()
90 7
91 8     def test_validate_singleton_instance(self):
92         first_instance = ClinicDatabase.get_instance()
93         second_instance = ClinicDatabase.get_instance()
94         self.assertIs(first_instance, second_instance)
95 9
96 10     def test_register_pet_owner(self):
97         owner_id = self.db.register_owner("Juan Dela Cruz", "09170000000")
98         owners = self.db.get_owners()
99
100         self.assertIn(owner_id, owners)
101         self.assertEqual(owners[owner_id]["name"], "Juan Dela Cruz")
102         self.assertEqual(owners[owner_id]["contact"], "09170000000")
103 11
104 12     def test_add_pet_record(self):
105         owner_id = self.db.register_owner("Maria Dela Cruz", "09180000000")
106         pet_id = self.db.generate_pet_id()
107
108         pet = PetFactory.create_pet(pet_id, "Dog", "Bantay", owner_id)
109         self.assertIsInstance(pet, Dog)
110
111         self.db.add_pet(pet)
112         pets = self.db.get_pets()
113
114         self.assertIn(pet_id, pets)
115         self.assertEqual(pets[pet_id].name, "Bantay")
116         self.assertEqual(pets[pet_id].pet_type, "Dog")
117         self.assertEqual(pets[pet_id].owner_id, owner_id)
118 13
119 14     def test_schedule_appointment(self):
120         apt_id = self.db.schedule_appointment("Bantay")
121         appointments = self.db.get_appointments()
122
123         self.assertIn(apt_id, appointments)
124         self.assertEqual(appointments[apt_id]["pet_name"], "Bantay")
125         self.assertEqual(appointments[apt_id]["status"], "Scheduled")
126 15
127 16     def test_cancel_appointment(self):
128         apt_id = self.db.schedule_appointment("Bantay")
129         is_cancelled = self.db.cancel_appointment(apt_id)
130
131         self.assertTrue(is_cancelled)
132         appointments = self.db.get_appointments()
133         self.assertEqual(appointments[apt_id]["status"], "Cancelled")
134 17
135 18 if __name__ == "__main__":
136     unittest.main()
137 19
```

Laboratory Report 4

Design Patterns and Unit Testing

Screenshots of Output

(Insert images here from the output)





Laboratory Report 4

Design Patterns and Unit Testing

File Edit Selection View Go Run Terminal Help

Ln 225, Col 1 Spaces: 4 UTF-8 CRLF Python Python 3.14.3

Explorer

cpe106l-lab4-vet-clinic

> __pycache__

.gitignore

clinic_data.json

database.py

main.py

pet_factory.py

README.md

test_clinic.py

Outline

Timeline

Problems Output Debug Console Terminal Ports

Terminal

Python: dat...

```
=== Veterinary Clinic Management System ===
a. Pet Owner Management
b. Pet Management (Factory implementation)
c. Appointment Management
d. Run Unit Tests
e. Exit

Enter your choice: c

Appointment Management
a. Schedule Appointment
b. View Appointments
c. Update/Cancel Appointment
Select option: a
Enter Pet Name: Coco
Appointment scheduled successfully with Auto-Generated ID: APT-0003

=== Veterinary Clinic Management System ===
a. Pet Owner Management
b. Pet Management (Factory implementation)
c. Appointment Management
d. Run Unit Tests
e. Exit

Enter your choice: c

Appointment Management
a. Schedule Appointment
b. View Appointments
c. Update/Cancel Appointment
Select option: b

Scheduled Appointments:
Appointment ID: APT-0001 | Pet Name: Bantay | Status: Cancelled
Appointment ID: APT-0002 | Pet Name: Bantay | Status: Scheduled
Appointment ID: APT-0003 | Pet Name: Coco | Status: Scheduled

=== Veterinary Clinic Management System ===
a. Pet Owner Management
b. Pet Management (Factory implementation)
c. Appointment Management
d. Run Unit Tests
e. Exit

Enter your choice: c

Appointment Management
a. Schedule Appointment
b. View Appointments
c. Update/Cancel Appointment
Select option: a
Enter Pet Name: Coco
Appointment scheduled successfully with Auto-Generated ID: APT-0003

=== Veterinary Clinic Management System ===
a. Pet Owner Management
b. Pet Management (Factory implementation)
c. Appointment Management
d. Run Unit Tests
e. Exit

Enter your choice: c

Appointment Management
a. Schedule Appointment
b. View Appointments
c. Update/Cancel Appointment
Select option: b

Scheduled Appointments:
Appointment ID: APT-0001 | Pet Name: Bantay | Status: Cancelled
Appointment ID: APT-0002 | Pet Name: Bantay | Status: Scheduled
Appointment ID: APT-0003 | Pet Name: Coco | Status: Scheduled

=== Veterinary Clinic Management System ===
a. Pet Owner Management
b. Pet Management (Factory implementation)
c. Appointment Management
d. Run Unit Tests
e. Exit

Enter your choice: 
```

Ln 225, Col 1 Spaces: 4 UTF-8 CRLF Python Python 3.14.3

19:17 9/12/2026

File Edit Selection View Go Run Terminal Help

Ln 225, Col 1 Spaces: 4 UTF-8 CRLF Python Python 3.14.3

Explorer

cpe106l-lab4-vet-clinic

> __pycache__

.gitignore

clinic_data.json

database.py

main.py

pet_factory.py

README.md

test_clinic.py

Outline

Timeline

Problems Output Debug Console Terminal Ports

Terminal

Python: dat...

```
b. Pet Management (Factory implementation)
c. Appointment Management
d. Run Unit Tests
e. Exit

Enter your choice: c

Appointment Management
a. Schedule Appointment
b. View Appointments
c. Update/Cancel Appointment
Select option: a
Enter Pet Name: Coco
Appointment scheduled successfully with Auto-Generated ID: APT-0003

=== Veterinary Clinic Management System ===
a. Pet Owner Management
b. Pet Management (Factory implementation)
c. Appointment Management
d. Run Unit Tests
e. Exit

Enter your choice: c

Appointment Management
a. Schedule Appointment
b. View Appointments
c. Update/Cancel Appointment
Select option: b

Scheduled Appointments:
Appointment ID: APT-0001 | Pet Name: Bantay | Status: Cancelled
Appointment ID: APT-0002 | Pet Name: Bantay | Status: Scheduled
Appointment ID: APT-0003 | Pet Name: Coco | Status: Scheduled

=== Veterinary Clinic Management System ===
a. Pet Owner Management
b. Pet Management (Factory implementation)
c. Appointment Management
d. Run Unit Tests
e. Exit

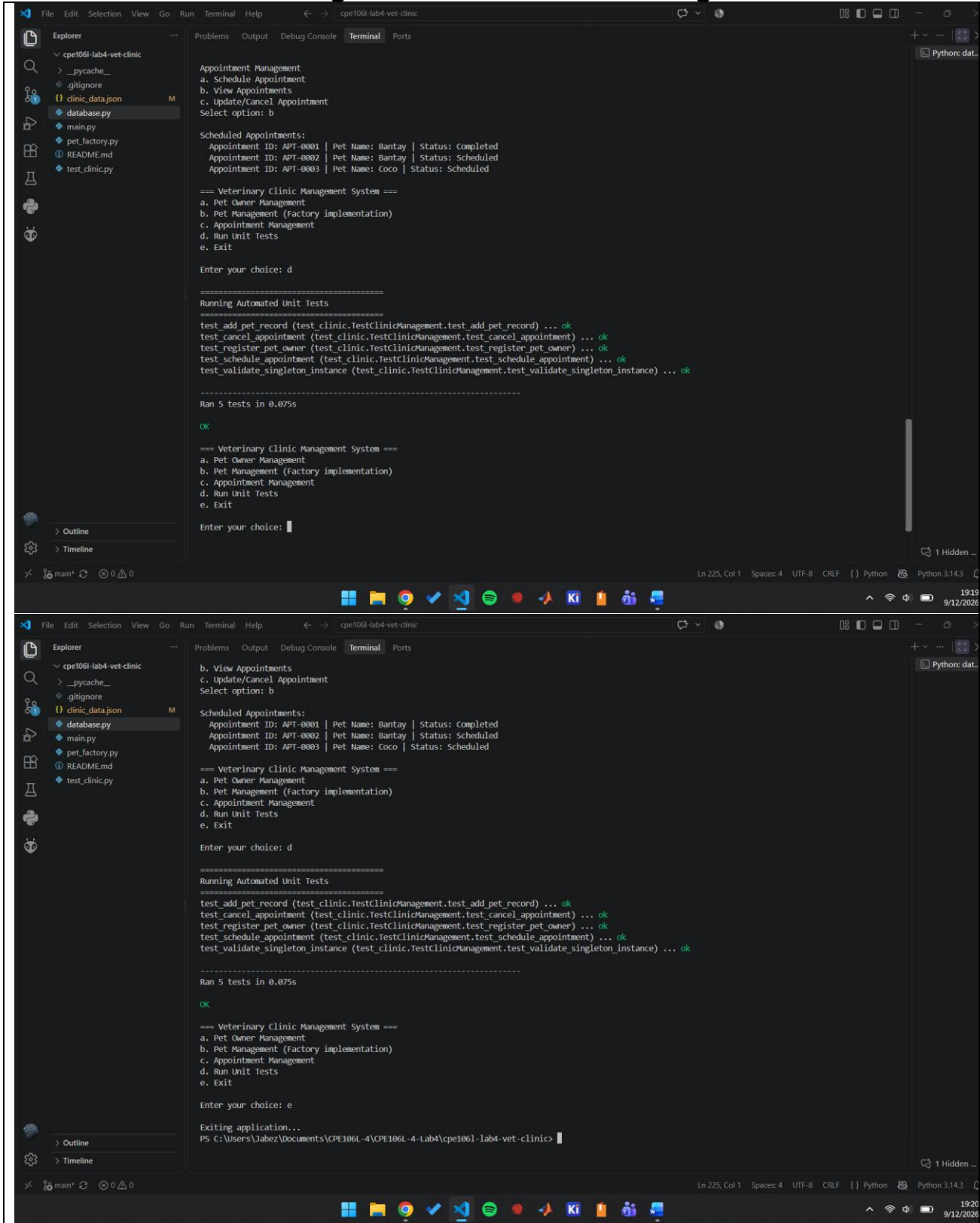
Enter your choice: 
```

Ln 225, Col 1 Spaces: 4 UTF-8 CRLF Python Python 3.14.3

19:18 9/12/2026

Laboratory Report 4

Design Patterns and Unit Testing



*Resize the image if necessary.

Design Patterns and Unit Testing**D. Findings, Observations, and Comments:**

Guide Questions:

1. What are your Group's realizations in this laboratory?
2. What are the advantages of using design patterns in software development?
3. What challenges did you face during testing phases?

(Please type your findings, observations, and comments according to the guide questions in a paragraph format, Tahoma; 10)

Our group realized that design patterns help make software more organized and easier to maintain. We learned how the Singleton and Factory patterns can be applied to solve specific programming problems. We also realized that unit testing is important for checking if the system works correctly.

Design patterns provide reusable solutions to common software development problems. They make the code easier to understand, maintain, and modify. In our project, they helped manage the clinic database and create different pet types efficiently.

One challenge we faced was ensuring that all functions worked correctly during testing. We encountered some errors in registering owners, adding pets, and managing appointments. Through debugging and repeated testing, we were able to identify and fix these issues.

PICTURE/PROOF OF DOING THE ACTIVITY



Laboratory Report 4

Design Patterns and Unit Testing

E. Score Sheet

Criteria	Poor (25%)	Fair (50%)	Good (75%)	Excellent (100%)	Score
I. Completeness, Documentation, and Organization of Report	The report was incomplete, messy, and had many unanswered questions.	The report was complete, messy, and had many unanswered questions.	The report was complete, neat, and had 1 or 2 unanswered questions.	The report was complete, neat, and all the questions had been answered.	
II. Coding Design and Patterns	The program had inappropriate elements and structures, incorrect indentation, and poor program documentation.	The program had corrected indention, but inappropriate elements and structures and poor program documentation.	The program had corrected indention, adequate program documentation, but inappropriate elements and structures.	The program had appropriate elements and structures, correct indention, and excellent program documentation.	
III. Findings, Observations, and Comments	The findings. observations, and comments were not based on the gathered data and results. All thoughts were inconsistent and unclear.	The findings. observations, and comments based on the gathered data and results but were not elaborated. Not all thoughts were consistent and clear.	The findings. observations, and comments were based on the gathered data and results and were mostly elaborated. Most of the thoughts were consistent and clear.	The findings. observations, and comments were based on the gathered data and results and were completely elaborated. All the thoughts were consistent and clear.	
IV. Grammar and Composition	The words used were inappropriate, wrong grammar usage, and poor sentence construction was observed.	The words used were appropriate; however, bad grammar and poor sentence construction were observed.	The words used were appropriate, had proper grammar usage, but poor sentence construction was observed.	The words used were appropriate, had proper grammar usage, and excellent sentence construction.	
SCORE:					

CHECKED			
Rating		Signature	
		Date	